UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2023/2024

SEPTEMBER 2023 EXAMINATION

UECS3253 / UECS3463 WIRELESS APPLICATION DEVELOPMENT

FRIDAY, 29 SEPTEMBER 2023 TIME : 2.30 PM – 4.30 PM (2 HOURS)

BACHELOR OF SCIENCE (HONOURS) SOFTWARE ENGINEERING
BACHELOR OF ENGINEERING (HONOURS) ELECTRONICS
(COMPUTER NETWORKING)

Instructions to Candidates:

This question paper consists of **THREE (3)** questions in **Section A** and **TWO (2)** questions in **Section B**.

Section A: Answer **ALL THREE (3)** questions.

Section B: Answer **ONLY ONE (1)** question.

Each question carries 25 marks.

Remark: Should a candidate answer **TWO (2)** questions in **Section B**, marks will only be awarded for the **FIRST** question in the order the candidate submits the answers.

Answer the questions in the answer booklet provided.

Section A: Answer **ALL** questions

Q1. (a) There are three (3) types of mobile apps, i.e., native apps, mobile web apps and hybrid apps.

(i) Explain the difference between native apps and mobile web apps. (4 marks)

(ii) Consider that the user needs to use the apps even the wireless device when there is no internet connection, identify **ONE (1)** suitable type of mobile app. (2 marks)

(b) Android refers to an operating system (OS) in the wireless platforms.

(i) Android devices supported various features, such as data storage and messaging. In addition to the previously mentioned features, list **FOUR (4)** features in an Android device. (4 marks)

(ii) Identify the official language for Android apps development. (2 marks)

(iii) Other than Android, identify **ONE (1)** OS for wireless platforms. (1 mark)

(c) In React Native, component is one of the important factors during the application development process where anything displayed on the screen are components.

(i) Most of the components can be customized using props and state. Explain the difference between props and state. (4 marks)

(ii) Each component has a few lifecycle methods that can be overridden. Identify the prefix of the methods that are called right after something happens. (2 marks)

(iii) Constructor(props) refers to the constructor for a component and it is called before the component is mounted. Describe the reason that the super(props) should be called before any other statement in the constructor(props). (2 marks)

(iv) State **TWO (2)** usages of the constructor. (4 marks)

[Total : 25 marks]

Q2. (a) SQLite is one of the SQL database engines, and it is a relational database management system (RDBMS) in a C programming library.

(i) Describe the difference between SQLite and other database management systems. (2 marks)

(ii) `react-native-sqlite-storage` refers to a plugin for React Native to use SQLite, provide **THREE (3)** features of this plugin. (6 marks)

(b) Consider the **Table 2.1** named "`bookList`" with three columns: "ID" (integer), "`Title`" (text), and "`Price`" (real). Write a code segment that **creates** the "`bookList`" table in SQLite with the specified columns and **inserts ALL** the provided data into the table. (8 marks)

Table 2.1: `bookList`

ID	Title	Price
1	Mockingbird	RM 12
2	Pride and Prejudice	RM 8

(c) Complete the following code segment that retrieves all the books from the table in the SQLite. (4 marks)

```
// Assuming you have already established the SQLite
database connection as 'db'

queryBook() {
      this.db.transaction(
        /* Complete the code here*/
        ...
        ...
        );
}
```

(d) In a React Native app, you need to display the retrieved product records in a `FlatList` component. Assume the retrieved product records are stored in variable as shown below. Construct the code segment to render it in the `FlatList` with the book's title and price. (5 marks)

```
Books=[
  {"id" : "1",
   "name" : "Mockingbird",
   "price": "RM 12"
  },
  {"id" : "2",
   "name" : "Pride and Prejudice",
   "price": "RM 8"
  }]
```

Q2. (d) (Continued)

```
Render(){
return(
        <View style={StyleSheet.container}>
        <FlatList
        /* Complete the code here */
        ...
        ...
        ...
        >
        </FlatList>
        </View>
}
```

[Total : 25 marks]

Q3. WeatherSense is a location-based weather application developed by React Native. The app provides users with the current weather conditions based on their geolocation. To achieve this, it firstly needs to request location permission in React Native. Then, retrieve the user's geolocation.

(a) Explain **TWO (2)** different ways of requesting the location permission within a React Native for WeatherSense. (8 marks)

(b) Create a code segment that demonstrates the device's current location and gives back a position object with the data using the Geolocation API. (7 marks)

(c) `getCurrentPosition` and `watchPosition` are two important methods from the Geolocation module in React Native.

(i) Define the distinctions between the `getCurrentPosition` and `watchPosition` methods. (5 marks)

(ii) Create a code segment to show how `watchPosition` can be utilized in a React Native application. (5 marks)

[Total : 25 marks]

Section B: Answer ONLY **ONE (1)** question

Q4. You are developing a mobile app for a laboratory booking system that allows lecturers to schedule their practical classes. The app should provide the several functionalities for users, including Reservation Request and Lab Availability.

(a) **Reservation Request:** Lecturers should be able to send a reservation request to the server, specifying the desired lab room, date, and time for their practical class. The app will send the reservation details in `JSON` format, the information inside the `JSON` is listed in **Table 4.1**, to the server using the `HTTP POST` method.

Table 4.1: Reservation Request

Field	Type	Data	Description
"lecturer"	String	Sarah Smit	The name of the lecturer making the reservation.
"labRoom"	String	606	The desired lab room for the practical class
"date"	String	14-06-2023	The date of the practical class
"time"	String	3:00 PM	The time slot for the practical class

Upon successfully submitting the reservation request, the server will respond with a confirmation message and a unique reservation ID in `JSON` format, and the information inside the `JSON` are shown in **Table 4.2**.

Table 4.2: Reservation Response

Field	Type	Description
"reservationID"	String	The unique ID assigned to the reservation
"message"	String	A confirmation message indicating success
"success"	Boolean	A flag indicating the success of the reservation

Implement a code segment that sends the reservation request data to the server at the URL `https://booking.labapp.com/api/reserve` using the provided information. If the reservation is successful, display a message to inform the user and navigate them to a screen named `"ReservationDetails"` while passing the reservation ID as a route parameter. In case of an error during the connection or communication with the server, display an appropriate error message to the user. (14 marks)

(b) **Lab Availability:** The app should provide a feature that allows lecturers to check the availability of lab rooms for a specific date and time. The server will respond with a `JSON` data object containing only the lab room numbers which are available, and the information inside the `JSON` as shown in **Table 4.3**.

Table 4.3: Lab Availability Response

Field	Description
"lab_room_1"	Availability status of Lab Room 1 for the given time
"lab_room_2"	Availability status of Lab Room 2 for the given time
...	...

Q4. (b) (Continued)

Implement a code segment that sends a query for lab availability to the server using the `HTTP GET` method at the URL `https://booking.labapp.com/api/availability`. If any lab rooms are available, navigate the user to a screen named "`AvailableLabs`" and pass the availability data as route parameters. Otherwise, display an alert with an appropriate error message indicating that no lab rooms are available. Handle any errors that may occur during the connection or communication with the server by displaying an appropriate error message. (11 marks)

[Total : 25 marks]

Q5. SmartHotel is a mobile application that allows users to make real-time hotel reservations. The app establishes a TCP socket connection to the server at `smarthotel.server1.my:8080` to communicate with the backend. The server provides information about room availability in response. Upon receiving the reservation, the server responds with a success status.

(a) Develop a code segment in React Native that utilizes the WebSocket API to establish a TCP socket connection with the server at `smarthotel.server1.my:8080/reserve`. The reservation details, as shown in the **Table 5.1** (in `JSON` format), should be sent to the server. A message will pop up to notify the user when the connection is established. Handle the response data from the server, as shown in **Table 5.2** (in `JSON` format), by navigating the user to a screen named "`ReservationDetails`" and passing the received data as a route parameter. In case of any errors during the connection or communication with the server, display an appropriate alert message to the user.

Table 5.1: User's Reservation Details (`JSON` Format)

```
{
"name": " Sarah Smit ",
"email":       "       Sarah
@test.com",
"checkIn": "2023-06-22",
"checkOut": "2023-06-25",
"roomType": "Single",
"guests": 1
}
```

Table 5.2: Server Response Data (`JSON` Format)

```
{
"reservationID": "12345",
"confirmationNumber":
"ABC123456",
"totalPrice": "$500.00"
}
```

(14 marks)

Q5. (Continued)

(b) The app allows users to view their reservation history. To accomplish this, a request will be sent to the server from the app. Implement a code segment in React Native that firstly connects to the server at `smarthotel.server1.my:8080/history` via the WebSocket API. Then, retrieve the user's reservation history by sending a request to the server. Upon receiving the response, display the reservation history on a screen named "`ReservationHistory`" in the app. The server will respond with an array of reservation objects, as shown in **Table 5.3** (in `JSON` format).

Table 5.3: Server Response Data - Reservation History (`JSON` Format)

```
[{
"reservationID": "12345",
"checkIn": "2023-07-15",
"checkOut": "2023-07-20",
"roomType": "Deluxe",
"totalPrice": "$500.00"
},
{
"reservationID": "67890",
"checkIn": "2023-08-10",
"checkOut": "2023-08-15",
"roomType": "Standard",
"totalPrice": "$300.00"
},
...
]
```

Make sure the code segment handles errors that may occur during the connection or communication with the server. If an error occurs, display an appropriate alert message to the user. (11 marks)

[Total : 25 marks]